

Planning

We continued to use our Discord chat room to discuss and plan our project. To keep the project on track, we scheduled virtual voice meetings to synchronously discuss the project and our progress. Roles were assigned based on the UML class diagram from Phase 1, giving each member a clear area to own and reducing the chance of overlapping work.

Midway-Progress Goals

After dividing responsibilities initially, we planned to have the core parts of the game set up and working by the midway project deadline. By the time however, progress was limited. We had set up the Maven build system and gotten FXGL running as our initial game framework, but had not yet implemented substantial gameplay. Shortly after, we discovered FXGL was incompatible with the project requirements and switched to pure JavaFX, which meant restarting much of the early work. In hindsight, we did not meet our midway goals, but the pivot was necessary and development moved such faster once we had settled on the right tools.

Division of Responsibilities

We divided responsibilities into categories, based on their placement and connections in the UML diagram created in project phase 1.

- **Marcus** was responsible for the board, tile, and entity logic. This includes pathfinding for the enemy, player controls, and handling the integration/interactions between the game loop and the UI. He created the initial project layout, classes, and their relations (based on the definitions in the UML diagram). He enforced the boundary between game and render logic across the codebase, and also spearheaded the transition from FXGL to vanilla JavaFX.
- **Kenneth and Mu** shared responsibility for the game's UI design, audio, and board rendering.
 - **Kenneth** was responsible for the game's screen and UI implementation. This includes most game menus (pause, win, game over, tutorial), the in-game HUD overlay displaying the timer, score and data packet progression, and most visual feedback effects. He also handled font and resource optimization to reduce rendering lag.
 - **Mu** was responsible for the game's audio system, sprite artwork, board rendering, and camera control. This includes designing the player, enemy,

and tile sprites, implementing camera zoom to follow player, and implementing the audio manager class for sound effects.

- **Brandon** was responsible for the procedural generation of the game board and the design of the mechanics in the game. He designed the rooms, walls and was responsible for the logic of key in game mechanics like the key system to open the server room, the firewall spreading, the freezing mechanic, and the algorithm to place random shaped wall clusters. Additionally, some bugs and performance issues were directly solved by him.

Implementation

The UI layer is structured around the `Menu` base class, with each screen extending it. Screen transitions are handled through `HackingGame.openMenu()`, which manages the open/close lifecycle of each screen. Screens include the title, options, in-game, pause, win, and lose.

The in-game view is managed by `GameMenu`, which implements `BoardObserver` to receive state change notifications from the board thread and synchronize them to the JavaFX render thread via `Platform.runLater()`. On each board update, `GameMenu` refreshes the positions of entities, updates the camera, and updates the HUD values.

The camera is implemented in `GameMenu` as a group that contains all entities. This grouped layer is scaled by a zoom factor, and as the player moves, the world layer is translated in the opposite direction to keep the player centred on the screen. The `Hud` class is a separate overlay composited independently on top of the game view, keeping rendering concerns modular and prevents game-world movement from affecting overlay elements.

Design Changes

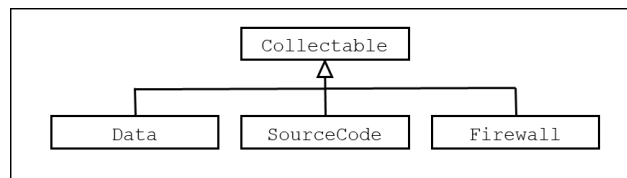
The phase one UML diagram envisioned manual `render()` methods on entities, tiles, and menus to board. It was designed to be used with a lower-level immediate-mode rendering framework like LibGDX or LWJGL. Since we went with the JavaFX graphics library (which is not immediate-mode, but retained-mode), this had to change. Now, entities and tiles are graphically represented **using JavaFX Nodes**, whose rendering is managed by the JavaFX runtime. Menus are composed of these nodes and can use the library's buttons and other utilities rather than manually checking for mouse/keyboard input.

When implementing the board, we realized that most tiles are simple enough that they do not need to have their own state or identity. For example, two floor tiles are functionally identical except for their position. With this knowledge, we made each tile represent a

“type” of tile that followed the **Singleton pattern** and made each method take a board state and position, rather than attaching it to an individual tile instance. This simplified board construction and saved some memory (as we no longer had to create a new tile for every position on the board).

```
public abstract class TileType {  
    public void onStep(Board board, Position position, Entity entity) {
```

The class hierarchy of the UML diagram specified separate Reward and Enemy classes, with the normal/bonus rewards and static/moving enemies subclassed, respectively. When implementing the entities, we found that the static enemy’s functionality is very closely related to the bonus/normal rewards, while the moving enemy was different. So, we **replaced the Reward class with Collectable** (now supporting both positive and negative score changes) and subclassed the static enemy from that. This allowed us to de-duplicate the collision handling and score logic between these types. Much of the room types in the UML diagram had to be scrapped due to lack of functionality and use, instead focusing on a key system that allowed the server room to be accessed only through finding the key in the storage room. A freezing mechanic was then added as an entity in the server room as a reward for completing this interaction.



Regarding the use cases, most of the use cases were implemented and follow the game logic, sometimes using different wording and slight changes. One of these changes is in use case 4 where the game does not immediately end upon obtaining all data packets, you must first get to the exit yourself. Another minor change is in use case 8 that tells the reader there will be an Options tab in the pause menu. This tab now only exists from the title screen. The biggest change is the removal of customization of display settings, control keys, and graphics quality in said Options tab. It was not needed for this caliber of game so you may only adjust the sound.

The room and board generation system was expanded significantly beyond the original design. The initial plan described a simple grid; the final implementation includes distinct named rooms (a server room and storage room), procedurally generated interior walls each run, a firewall spread mechanic, and a room key system that was not present in the original design. The enemy movement frequency was adjusted from every tick to every two ticks for balance and performance reasons.

External Libraries

Our sole external library is JavaFX, which we added to handle graphics, input, and audio. We chose it because it's a relatively simple, pure Java graphics library. There was an intermediate step where we tried using the FXGL game library, but we found that it handled too many things like the game world, and followed an entity component system, incompatible with our object-oriented design, so we switched to pure JavaFX. We also evaluated LibGDX but found that it did not support the Maven build system.

Measures taken to enhance quality of code

To enhance the quality of code, we separated the game (board) logic from the rendering of the game. Instead of having the board extend a JavaFX Pane, and all entities extend `Node`, we kept these concepts separate. This allowed us to have all game logic run asynchronously on a separate thread, instead of interrupting the render thread. It also had the additional benefit of making the game more testable, as we have full control of the game's update loop and don't need to initialize JavaFX to simulate the game. At the end of each game loop, we synchronize the board state to the render thread.

We also made sure to document classes and methods with JavaDoc so that interactions between components are unambiguous. This practice has improved the code's readability and made collaboration easier.

Challenges

We originally attempted to use FXGL as our game library, but as stated earlier we found that it handled too many things and was incompatible with the project criteria: FXGL did not properly support testing with JUnit, and control of the game loop was hidden within the library. This required us to rapidly shift away from FXGL and to pure JavaFX to still meet the project deadline. It was frustrating to have to take this step late into the project's development, but it had to be done.

Another challenge was managing the boundary between the game logic thread and the JavaFX render thread. The board needs to update at a fixed interval while JavaFX renders the UI on its own timeline, so we couldn't just run game logic every X frames. Instead, we needed to run all the game logic on a separate thread and then synchronize it to the screen. JavaFX requires all UI modifications to happen on its own thread, so synchronization operations that forgot to execute in the JavaFX context (via `Platform#runLater`) caused intermittent crashes that were difficult to reproduce.

Performance was also a concern in the UI layer. Early implementations of the animated sprites and HUD elements had noticeable lag spikes. We eventually found the problem to be these elements loading font files and asset images from disk on every frame update. This was resolved by pre-loading fonts and images once at construction time, storing them as `Image` and `Font` objects in a hash map, and reusing the cached instances.

A similar issue was found in audio playback, where the first playback after starting the game would be significantly delayed, and the subsequent playbacks are normal. This issue remained after preloading the audio files, and we found the problem to be the one-time initialization of the library's audio engine. We mitigated this issue by playing a silent audio while the game initializes, so that the first "real" playback is ready.

Additionally, there was a design dilemma for how the map would be generated to create natural, random obstacles without blocking access to either special room. The `generateWall()` method chooses between three different shape types: a single tile, a 2-tile path (horizontal or vertical), and a 2x2 square. For each shape, a random position on the map is picked first, then the shape is created from that point. Each tile must pass through `placeWall()` which acts as a safety filter. This filter will reject any tile that falls inside either special rooms, on a doorway, or is within a 1-tile buffer zone outside the rooms boundary walls. The `isSolid()` check also ensures that shapes are not overwriting tiles that are already walls.

There was also the question of pacing of the game. The point of this game was to be a high-speed chase through digital corridors, but we also needed to make sure the firewall spreading mechanic was not too fast of a spread as to be overwhelming, and not too slow to be obsolete. A balance was needed between the spawn rate of the source code and how fast the fire spread. This is the reason why the freezing item was implemented. It gives the user time to collect more source codes while the spread freezes momentarily.